

Week 2 Module

By (Mohit Sharma), 2404921540131

Problem-1:

1. Fraud Transaction Detection (Linear Search)

Problem Statement

A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists.

Write a program using **Linear Search**.

Input Format

- First line contains integer N
- Second line contains N transaction IDs
- Third line contains suspicious transaction ID

Output Format

Print:

Fraud Transaction Found

or

Transaction Not Found

Example

Input

```
5
1201 1305 1450 1408 1502
1450
```

Solution:

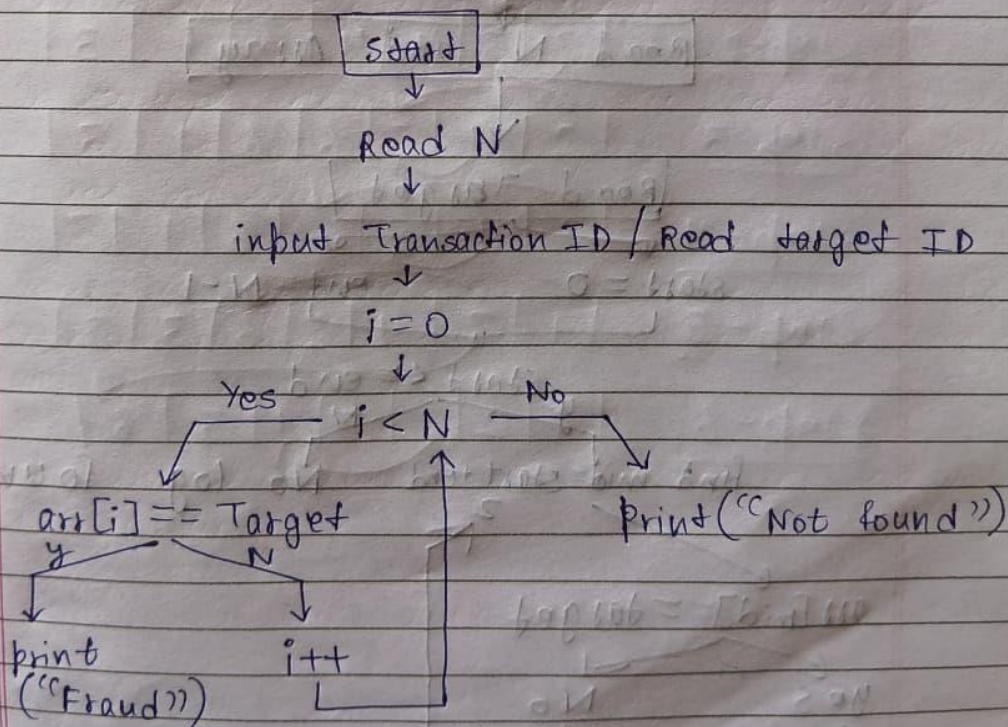
Week 2 Module

Experiment - 1

→ Fraud detection of Transaction using linear search

Linear search is the simplest search algorithm technique.

- checks each element until the end
- when data is unsorted
- dataset is small
- simplicity is preferred.



```
Week 2 Module.cpp X ... bash X
Week 2 Module.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cout<<"Enter the number of transactions: ";
8      cin >> n;
9
10     int arr[n];
11     cout<<"Enter the transaction IDs: ";
12     for(int i = 0; i < n; i++)
13     {
14         cin >> arr[i];
15     }
16
17     int target;
18     cout<<"Enter the transaction ID to search: ";
19     cin >> target;
20
21     bool found = false;
22     for(int i = 0; i < n; i++)
23     {
24         if(arr[i] == target)
25         {
26             found = true;
27             break;
28         }
29     }
30
31     if(found)
32         cout << "Fraud Transaction Found";
33     else
34         cout << "Transaction Not Found";
35
36     return 0;
37 }
```

```
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$ g++ Week\ 2\ Module.cpp -o main.exe && ./main.exe
Enter the number of transactions: 5
Enter the transaction IDs: 10
40
50
30
60
Enter the transaction ID to search: 40
Fraud Transaction Found
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$
```

Problem -2:

2. Server Log Search (Binary Search)

Problem Statement

A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists.

Write a program using **Binary Search**.

Input Format

- First line contains N
- Second line contains sorted log IDs
- Third line contains target log ID

Output Format

Print:

Log Found

or

Log Not Found

Constraints

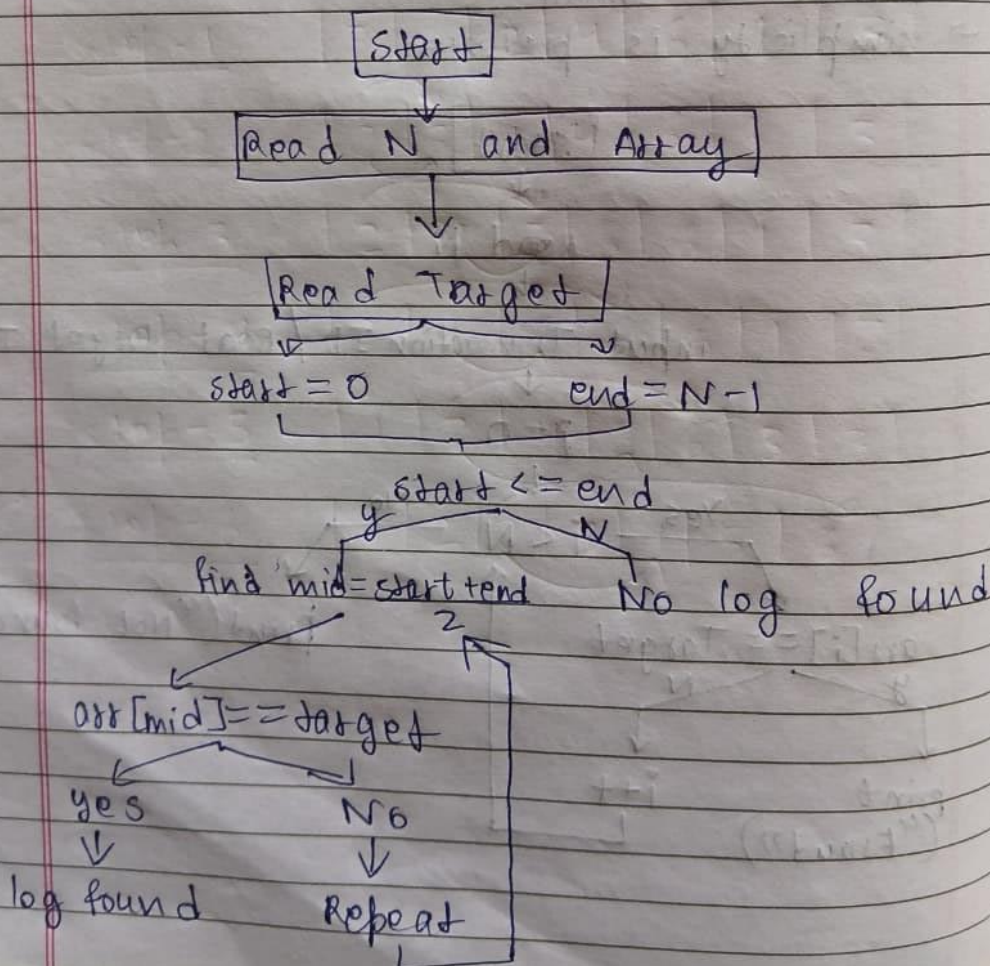
- $1 \leq N \leq 10^5$

Solution:

Problem - 2

→ Server log search using binary search

Binary search is an efficient searching algorithm used to find an element in a sorted array. It compares the target element with middle element of the array. If they are equal, the search is complete. otherwise it continues searching.



```
Week 2 Module.cpp X ... bash X
Week 2 Module.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      int n;
6      cout<<"Enter the number of logs: ";
7      cin >> n;
8      int arr[n];
9      cout<<"Enter the log IDs in sorted order: ";
10     for(int i = 0; i < n; i++){
11         cin >> arr[i];
12     }
13
14     int target;
15     cout<<"Enter the log ID to search: ";
16     cin >> target;
17
18     int start = 0;
19     int end = n - 1;
20     bool found = false;
21
22     while(start <= end)
23     {
24         int mid = start + (end - start) / 2;
25         if(arr[mid] == target){
26             found = true;
27             break;
28         }else if(target < arr[mid]){
29             end = mid - 1;
30         }else{
31             start = mid + 1;
32         }
33     }
34
35     if(found) cout << "Log Found";
36     else cout << "Log Not Found";
37
38     return 0;
39 }
```

```
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$ g++ Week\ 2\ Module.cpp -o main.exe && ./main.exe
Enter the number of logs: 5
Enter the log IDs in sorted order: 45
67
34
68
79
Enter the log ID to search: 34
Log Found
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$
```

Problem-3

3. Sort Employee Salaries (Bubble Sort)

Problem Statement

A company wants to sort employee salaries in ascending order for payroll analysis.

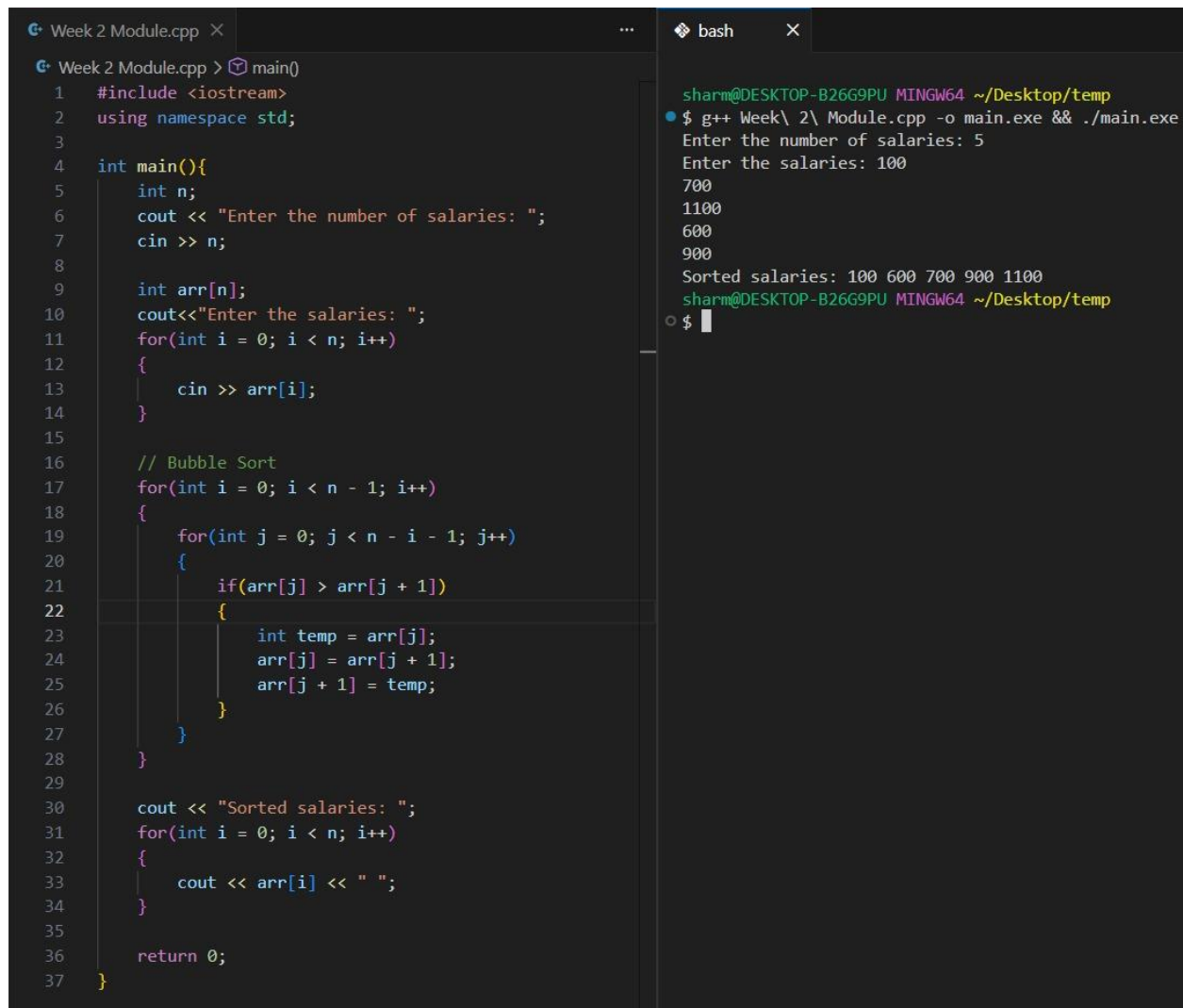
Write a program using **Bubble Sort**.

Input Format

- First line contains N
- Second line contains salaries

Output Format

Print sorted salaries.



```
Week 2 Module.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      int n;
6      cout << "Enter the number of salaries: ";
7      cin >> n;
8
9      int arr[n];
10     cout<<"Enter the salaries: ";
11     for(int i = 0; i < n; i++)
12     {
13         cin >> arr[i];
14     }
15
16     // Bubble Sort
17     for(int i = 0; i < n - 1; i++)
18     {
19         for(int j = 0; j < n - i - 1; j++)
20         {
21             if(arr[j] > arr[j + 1])
22             {
23                 int temp = arr[j];
24                 arr[j] = arr[j + 1];
25                 arr[j + 1] = temp;
26             }
27         }
28     }
29
30     cout << "Sorted salaries: ";
31     for(int i = 0; i < n; i++)
32     {
33         cout << arr[i] << " ";
34     }
35
36     return 0;
37 }
```

```
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$ g++ Week\ 2\ Module.cpp -o main.exe && ./main.exe
Enter the number of salaries: 5
Enter the salaries: 100
700
1100
600
900
Sorted salaries: 100 600 700 900 1100
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$
```

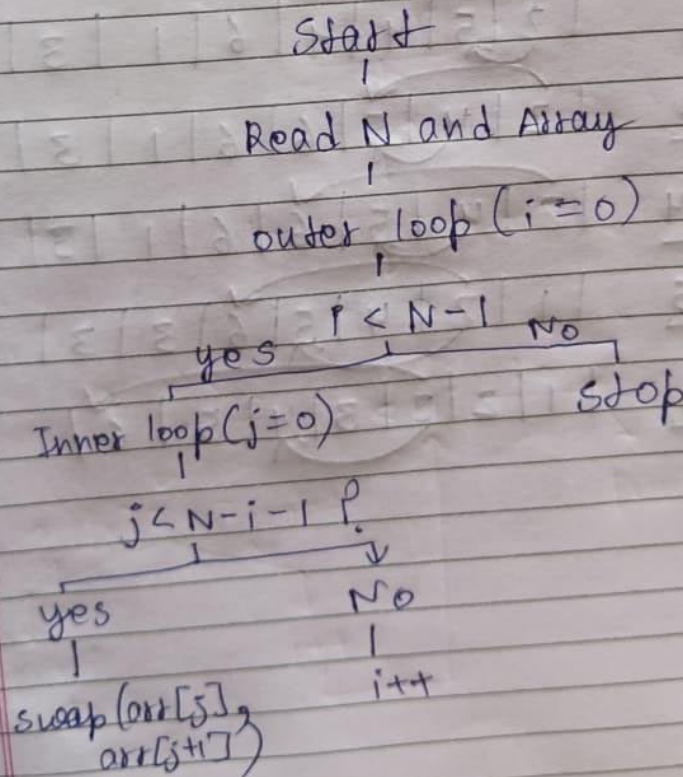
Notes....Problem-3:

Problem statement - 3

→ Sort employee salaries in ascending order.

- Bubble sort is a simple sorting algorithm that arranges elements in ascending or descending order by repeatedly comparing adjacent elements. If two adjacent elements are in wrong order, they are swapped.

This process continues until no more swaps are needed, indicating that array is sorted.



Problem-4:

4. Sort Website Response Time (Insertion Sort)

Problem Statement

A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position.

Write a program using **Insertion Sort**.

Input Format

- First line contains N
- Second line contains response times

Output Format

Print sorted response times.

Example

Input

```
5
250 120 320 180 100
```

Output

```
100 120 180 250 320
```

Solution:

Problem statement - 4

→ Insertion sort is simple sorting algo that builds the sorted array one element at a time. It picks one element from unsorted part and inserts it into its correct position in sorted part of the array.

→ efficient for small datasets

→ Adaptive

→ $O(n^2)$ time complexity

step-1 | 5 | 5 | 2 | 4 | 6 | 1 | 3 |

step-2 | 2 | 5 | 4 | 4 | 6 | 1 | 3 |

step-3 | 2 | 4 | 5 | 6 | 6 | 1 | 3 |

step-4 | 2 | 4 | 5 | 6 | 6 | 1 | 3 |

step-5 | 1 | 2 | 4 | 5 | 6 | 3 | 3 |

step-6 | 1 | 2 | 2 | 3 | 4 | 5 | 6 |

Week 2 Module.cpp X

...

bash X

Week 2 Module.cpp > main()

```
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      int n;
6      cout<< "Enter the number of elements: ";
7      cin >> n;
8
9      int arr[n];
10     cout << "Enter the elements: ";
11     for(int i = 0; i < n; i++){
12         cin >> arr[i];
13     }
14
15     for(int i = 1; i < n; i++){
16         int key = arr[i];
17         int j = i - 1;
18
19         while(j >= 0 && arr[j] > key){
20             arr[j + 1] = arr[j];
21             j--;
22         }
23
24         arr[j + 1] = key;
25     }
26
27     cout << "Sorted elements: ";
28     for(int i = 0; i < n; i++){
29         cout << arr[i] << " ";
30     }
31
32     return 0;
33 }
```

sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp

\$ g++ Week\ 2\ Module.cpp -o main.exe && ./main.exe

Enter the number of elements: 5

Enter the elements: 4

6

2

4

8

Sorted elements: 2 4 4 6 8

sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp

\$

Problem-5:

5. Prioritize Bug Reports (Selection Sort)

Problem Statement

A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order.

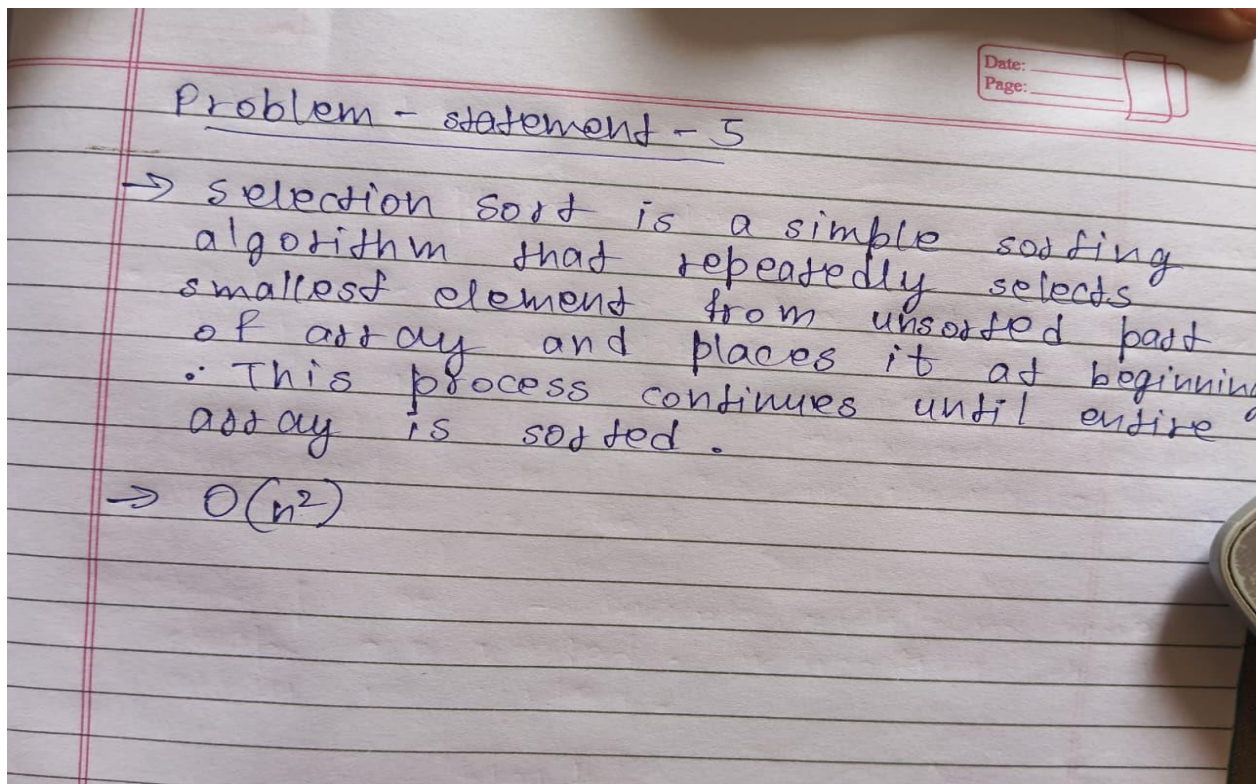
Write a program using **Selection Sort**.

Input Format

- First line contains N
- Second line contains priority scores

Output Format

Print sorted priority scores.



Code...Problem-5:

Week 2 Module.cpp X

...

bash X

Week 2 Module.cpp > main()

```
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      int n;
6      cout << "Enter the number of elements: ";
7      cin >> n;
8
9      int arr[n];
10     cout << "Enter the elements: ";
11     for(int i = 0; i < n; i++){
12         cin >> arr[i];
13     }
14
15     for(int i = 0; i < n - 1; i++){
16         int min = i;
17
18         for(int j = i + 1; j < n; j++){
19             {
20                 if(arr[j] < arr[min])
21                 {
22                     min = j;
23                 }
24             }
25
26             int temp = arr[i];
27             arr[i] = arr[min];
28             arr[min] = temp;
29         }
30
31         cout << "Sorted elements: ";
32         for(int i = 0; i < n; i++)
33         {
34             cout << arr[i] << " ";
35         }
36
37         return 0;
38     }
```

sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp

\$ g++ Week\ 2\ Module.cpp -o main.exe && ./main.exe

Enter the number of elements: 5

Enter the elements: 4

3

6

8

9

Sorted elements: 3 4 6 8 9

sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp

\$